

Semantic-Structural Robust Propagation for GNNs

Internal Research Memo
Working draft for discussion

Abstract

Internal technical memo. This document records the current problem framing, threat model, RUNG backbone, semantic trust mechanisms, and preliminary results for discussion.

1 Problem and Motivation

Consider node classification on a text-attributed graph $G = (V, E)$. A standard message-passing layer updates a node by aggregating messages from its observed neighbors:

$$h_i^{(\ell+1)} \leftarrow \text{Update}\left(h_i^{(\ell)}, \text{Aggregate}(\{h_j^{(\ell)} : j \in N(i)\})\right).$$

This computation assumes that the observed neighborhood provides reliable evidence. In practice, graphs can contain noisy, incorrect, or adversarially injected edges that transport misleading messages.

At each robust message-passing update, the model must make two decisions:

1. How much influence should the aggregate neighborhood message have?
2. How should that influence be distributed across individual neighbors?

To make these decisions on an unreliable graph, the model has access to two different sources of evidence: graph-internal dynamic evidence, such as topology and evolving node representations, and graph-external relation semantics, inferred from original node texts. Structural evidence alone can be fooled by camouflaged edges, while semantic evidence alone cannot fully characterize the current propagation state.

Existing robust GNNs typically answer both decisions using graph-internal signals, such as topology, feature similarity, or evolving representations. Our current v3 design makes one explicit division of labor: RUNG uses dynamic graph-internal evidence to estimate the overall neighborhood influence, while the LLM uses graph-external relation semantics to allocate that influence across neighbors.

Research question. How can we combine structural robustness and relation semantics for reliable message passing on unreliable graphs?

2 Threat Model: Text-Camouflaged Edge Injection

Attacker capability. The attacker knows the victim model and can add edges before evaluation, but cannot modify node texts.

Candidate generation. The attacker first retrieves textually related non-neighbor pairs, then uses white-box projected gradient descent (PGD) to select the subset of edges that maximally degrades node classification.

Key asymmetry. The attack corrupts graph structure, while the original node texts remain unchanged.

3 Experimental Setting

Task. We perform node classification on text-attributed graphs: Cora, Citeseer, and WikiCS. Each victim GNN is trained on the clean graph before attack.

BM25 candidate pool. For each node i , we use BM25 to rank all other node texts by lexical relevance to i . We retain the top- k nodes that are not already connected to i in the original graph. The union of these textually related non-neighbor pairs forms the candidate addition set C_{add} .

White-box PGD attack. The attacker knows the trained victim model, defense mechanism, and C_{add} . It assigns a continuous perturbation variable z_{ij} to each candidate edge in C_{add} , iteratively updates these variables along the gradient of classification loss, and finally injects the B highest-scoring edges.

Budget and evaluation. B is 5%, 10%, 20%, 30%, or 40% of the original edge count. The selected edges are added only at evaluation time; node texts remain unchanged. We report attacked node-classification accuracy.

BM25 constructs a pool of textually plausible non-edges; white-box PGD selects the most damaging subset for the victim GNN.

4 RUNG Robust Propagation Backbone

Initial representation and anchor. Each node is first encoded from its input feature:

$$f_i^0 = \text{MLP}(X_i).$$

The initial representation f_i^0 remains an anchor throughout propagation, so a node cannot be entirely pulled away by its neighborhood.

Dynamic edge consistency. Let $A_{ij} = 1$ when i - j is a currently observed edge and $A_{ij} = 0$ otherwise. Thus, after an attack, injected edges also appear in A . Let $d_i = \sum_j A_{ij}$ be the current degree of node i , and let $\tilde{A}_{ij} = A_{ij}/\sqrt{d_i d_j}$ be the usual degree-normalized adjacency coefficient. At propagation iteration t , RUNG measures the normalized squared discrepancy between endpoints of every current edge:

$$Z_{ij}^t = \left\| \frac{f_i^t}{\sqrt{d_i}} - \frac{f_j^t}{\sqrt{d_j}} \right\|^2, \quad d_{ij}^t = \sqrt{Z_{ij}^t}.$$

Small discrepancy means that the endpoints are currently compatible in the propagation state; large discrepancy indicates potentially inconsistent message passing.

MCP dynamic edge weighting. RUNG maps this discrepancy to a robust edge weight using the minimax concave penalty (MCP). MCP is a nonconvex robust penalty: it reduces the influence of increasingly discrepant edges and assigns zero influence once an edge is sufficiently discrepant. Its corresponding IRLS influence function, with a numerical stabilizer ϵ , is

$$u_{ij}^t = \sqrt{d_{ij}^t + \epsilon}, \quad W_{ij}^t = \begin{cases} \frac{1}{2u_{ij}^t} - \frac{1}{2\gamma}, & u_{ij}^t \leq \gamma, \\ 0, & u_{ij}^t > \gamma. \end{cases}$$

Our current implementation uses $\epsilon = 0.01$ and $\gamma = 36$. Compatible edges receive larger weights; highly inconsistent edges eventually receive zero weight and do not participate in that iteration.

Robust aggregation. For node i , the neighbor-message term is

$$\sum_j \tilde{A}_{ij} W_{ij}^t f_j^t.$$

It sums the current representations of all one-hop neighbors. $\tilde{A}_{ij}$ corrects for degree, while W_{ij}^t is RUNG’s current assessment of whether that edge should transmit a message. An edge with $W_{ij}^t = 0$ makes no contribution.

RUNG then combines this robust neighbor message with the node’s original representation f_i^0 :

$$\hat{Q}_i^t = \frac{\sum_j A_{ij} W_{ij}^t}{d_i} + \lambda, \quad f_i^{t+1} = \frac{\sum_j \tilde{A}_{ij} W_{ij}^t f_j^t + \lambda f_i^0}{\hat{Q}_i^t}.$$

Here λf_i^0 is an anchor term: even if the neighborhood becomes unreliable, node i retains a direct path to its own original feature-based representation. $\hat{Q}_i^t$ is the corresponding node-wise normalization (equivalently, the diagonal quasi-Newton curvature estimate), which keeps the relative scale of the weighted neighbor term and the anchor term stable as the degree and dynamic edge weights change.

The process repeats for ten QN-IRLS iterations. “IRLS” means that RUNG first uses the current representations to compute robust weights W^t , then treats these weights as fixed for one weighted propagation update. “QN” means that the update uses the diagonal quantity $\hat{Q}_i^t$ as a quasi-Newton preconditioner rather than solving a full global second-order system. Thus, W_{ij}^t is not a fixed preprocessing score: it is recomputed from the current propagation state at every iteration. These are ten solver iterations with shared mechanism, not ten unrelated GNN layers.

RUNG repeatedly compares the current representations of connected nodes, downweights edges that become inconsistent, and anchors every node to its original identity.

5 Semantic Trust Mechanisms

For every potential edge $i-j$, an LLM reads the endpoint texts and returns a fixed relation-semantic plausibility score q_{ij} . The LLM is evaluated offline; q_{ij} is frozen during GNN training and propagation.

5.1 v1: Edge-wise RUNG-LLM Multiplication

RUNG produces a dynamic structural weight W_{ij}^t for every edge and the LLM supplies a semantic score q_{ij} . v1 combines the two edge by edge:

$$\bar{W}_{ij}^t = c_i W_{ij}^t q_{ij},$$

where c_i is a normalization coefficient intended to avoid unconditional shrinkage of the total neighborhood weight. A low RUNG weight or a low LLM score reduces the final edge weight.

v1: RUNG and the LLM both score each edge, then their scores are multiplied.

5.2 v3: Semantic Trust Budgeting

RUNG still computes all dynamic edge weights W_{ij}^t , but v3 first retains only their sum for node i :

$$M_i^t = \sum_{j \in N(i)} W_{ij}^t.$$

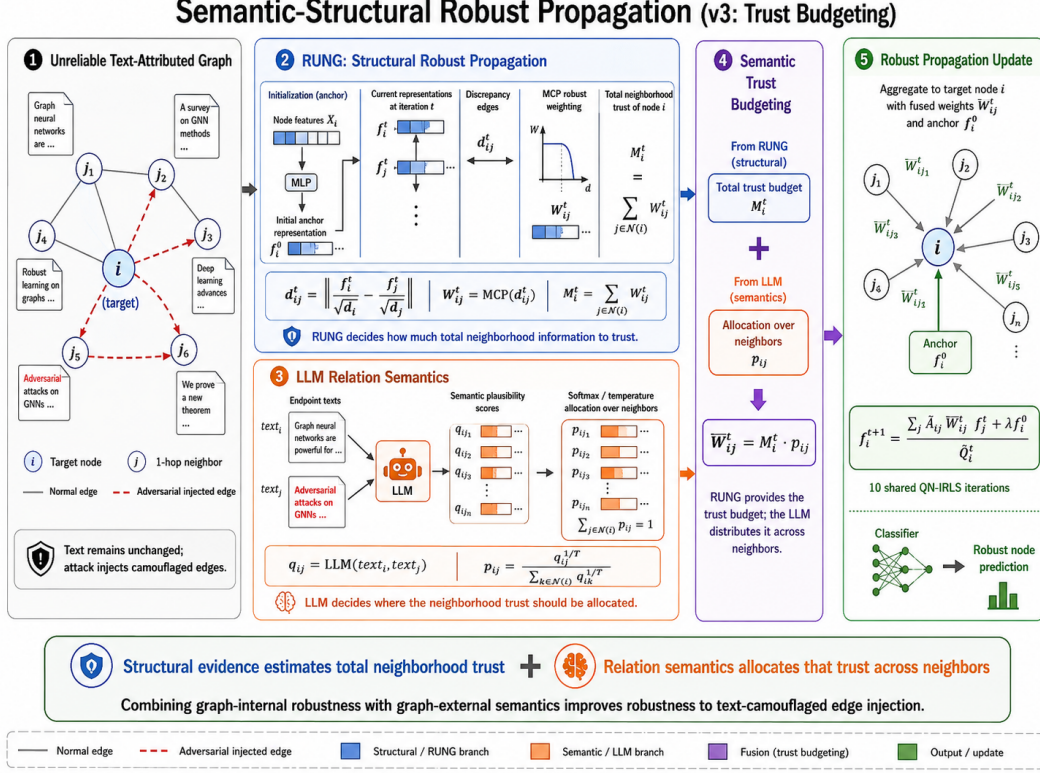


Figure 1: Method overview. RUNG uses dynamic graph-internal propagation evidence to estimate a node’s total neighborhood trust. The LLM reads endpoint texts to estimate relation semantics and allocate that trust across individual neighbors. The resulting weights are used for anchored robust propagation.

This is the current total neighborhood trust estimated from propagation behavior. The LLM scores are converted into an allocation over the current neighbors:

$$p_{ij} = \frac{q_{ij}^{1/T}}{\sum_{k \in N(i)} q_{ik}^{1/T}}.$$

The final propagation weight is

$$\bar{W}_{ij}^t = M_i^t p_{ij}.$$

Therefore, RUNG controls how much total information node i accepts from its current one-hop neighborhood, while the LLM controls how that trust is distributed among individual neighbors.

v3: RUNG determines the total neighborhood trust; the LLM determines its edge-wise allocation.

6 Current Results

All values below are mean $\pm$ standard deviation of node-classification accuracy (%) over 12 runs. “Clean” is the corresponding accuracy on the unperturbed graph. Bold denotes the highest mean in each attacked column.

6.1 Cora

Method	Clean	5%	10%	20%	30%	40%
GCN	83.3±0.9	75.1±0.8	70.8±1.2	65.0±1.8	61.3±3.1	58.1±3.5
GAT	82.9±0.4	75.8±0.3	71.1±0.6	64.9±0.7	60.8±0.4	57.6±0.9
APPNP	85.0±0.6	75.2±0.8	69.1±1.1	59.7±1.7	52.8±2.1	48.6±2.8
GCN-Jaccard	83.7±0.3	74.3±0.4	68.6±0.4	59.6±0.5	52.9±0.6	48.1±0.6
GNNGuard	83.3±0.6	76.1±1.0	71.0±0.6	63.1±1.1	57.0±0.9	51.8±0.5
RGCN	85.2±0.4	77.0±0.4	71.9±0.4	63.8±0.5	57.2±0.5	52.2±0.7
SCAD	83.6±0.4	73.0±1.2	65.0±2.3	54.2±3.0	45.9±3.0	41.7±4.0
TWIRLS	84.3±1.0	77.2±1.1	73.3±1.3	68.2±2.0	64.0±3.0	61.7±4.0
GraphSAGE	83.1±0.2	74.6±0.4	69.6±0.7	62.7±1.2	57.9±1.8	54.2±2.6
GRAND	83.0±0.4	71.9±0.6	65.3±1.1	54.0±1.2	44.9±1.3	36.9±1.5
H2GCN	82.8±0.3	74.6±0.2	70.4±0.2	64.7±0.8	60.6±1.4	57.6±1.7
RUNG	84.3±1.1	77.7±1.0	73.5±1.2	68.1±1.9	64.6±2.8	62.4±3.6
RUNG + cosine	82.7±1.3	75.2±1.5	71.1±1.4	65.1±2.4	63.0±2.7	60.5±4.2
RUNG + SBERT	83.1±1.2	76.6±1.2	73.4±1.1	69.0±1.5	66.1±2.2	64.1±3.0
v1: RUNG + LLM	83.4±1.1	77.3±1.2	74.3±1.5	70.4±2.0	68.6±2.8	66.5±3.0
v3: Trust Budget	82.7±0.9	79.6±1.2	77.7±1.4	75.0±1.9	73.2±2.5	71.8±3.1

6.2 Citeseer

Method	Clean	5%	10%	20%	30%	40%
GCN	74.3±1.1	67.5±1.2	63.9±1.6	58.4±2.2	54.9±3.0	52.2±3.9
GAT	74.6±0.6	70.8±1.1	67.4±1.1	63.3±2.1	60.4±3.0	57.4±4.0
APPNP	74.4±1.2	66.6±1.2	62.4±1.3	56.9±1.3	53.1±1.9	50.1±2.1
GCN-Jaccard	75.3±0.2	68.7±0.2	64.2±0.3	57.9±0.4	53.1±0.5	49.1±0.4
GNNGuard	75.2±0.4	69.9±0.6	65.6±0.6	59.3±0.8	54.5±0.7	50.8±0.9
RGCN	75.7±0.6	69.6±0.5	65.1±0.6	58.8±0.6	53.5±0.6	49.3±0.8
SCAD	73.2±1.0	65.3±1.4	61.2±1.7	55.6±2.2	51.8±2.4	48.6±2.6
TWIRLS	74.5±1.2	68.3±1.2	65.0±1.2	61.2±1.6	58.8±1.0	57.7±1.1
GraphSAGE	70.7±0.4	63.2±0.8	58.2±1.1	51.0±2.1	45.8±2.9	41.3±3.6
GRAND	69.3±0.8	59.7±0.6	53.5±0.6	44.6±1.1	37.9±1.6	31.9±1.8
H2GCN	71.5±0.4	66.2±0.8	63.1±1.6	59.4±1.4	56.8±2.0	55.0±1.8
RUNG	74.4±1.1	68.5±1.1	65.1±1.3	61.7±1.2	59.6±1.3	58.3±1.5
RUNG + cosine	74.1±0.9	68.2±1.0	65.2±1.1	61.2±1.7	58.9±2.4	57.2±3.2
RUNG + SBERT	74.6±1.0	69.2±0.9	66.1±1.1	62.5±1.3	60.5±1.8	59.0±2.1
v1: RUNG + LLM	74.5±1.4	69.4±1.5	66.9±1.6	63.7±2.1	61.6±2.5	60.4±2.9
v3: Trust Budget	73.6±0.9	71.5±0.9	70.2±0.9	68.6±1.0	67.3±1.1	66.3±1.2

6.3 WikiCS

Method	Clean	5%	10%	20%	30%	40%
GCN	79.3±0.7	61.5±1.0	58.7±1.3	56.6±1.1	55.2±1.3	54.7±1.1
APPNP	79.4±0.7	66.5±0.9	64.2±0.9	62.3±1.1	61.5±1.2	61.3±1.0
SCAD	78.4±0.7	62.3±1.5	58.8±1.7	55.5±1.6	54.0±2.0	52.9±1.9
TWIRLS	79.2±0.7	70.2±0.9	68.3±0.8	66.7±1.0	65.8±0.9	65.4±1.1
GraphSAGE	79.3±0.6	68.1±1.0	65.0±1.1	62.1±1.4	60.8±1.5	59.7±1.4
GRAND	79.0±0.5	63.2±1.2	58.6±1.1	54.0±1.2	51.6±1.3	49.9±1.2
H2GCN	79.7±0.3	70.1±0.8	67.8±1.0	65.6±1.1	64.6±0.7	63.7±1.0
RUNG	79.1±0.7	70.2±0.8	68.5±0.6	66.9±0.9	66.0±1.1	65.5±1.1
RUNG + cosine	79.3±0.6	70.2±0.9	68.4±1.0	66.6±1.1	66.1±1.2	65.6±1.3
RUNG + SBERT	79.7±0.7	71.1±0.8	69.4±0.8	68.0±0.9	67.5±0.7	66.9±0.9
v1: RUNG + LLM	80.1±0.5	74.0±0.7	72.6±0.6	71.4±0.7	70.9±0.9	70.7±0.8
v3: Trust Budget	79.2±0.6	75.7±0.6	75.0±0.6	74.3±0.8	74.0±0.8	73.7±0.8

7 Current Takeaways

The current v3 RUNG plus LLM trust-budget mechanism is strongest at every evaluated attack budget on all three datasets. It trades a modest amount of clean accuracy on Cora and Citeseer for substantially stronger attacked accuracy. The present scope is text-camouflaged, add-only, white-box edge injection; scalability and broader evaluation remain open questions.

8 Next Steps

Direction 1: Method. Understand and refine every component of the current RUNG plus LLM-based semantic trust-budget mechanism; identify and reduce the clean-accuracy cost

of v3; and develop a path to large graphs, including lower-cost semantic scoring and teacher-student distillation.

Direction 2: Evaluation. Systematically review recent robust-GNN baselines, identify the strongest missing comparisons, and investigate more realistic robustness settings where the combination of graph-internal structural evidence and external relation semantics is genuinely necessary.

Related, separate investigation. A parallel, self-contained investigation on the STEM-GNN pretrain/finetune codebase (a different backbone and evaluation protocol from the RUNG/LLM setting here) explores a related but distinct question: bounding the Lipschitz constant of the GNN encoder itself, rather than combining structural and semantic trust signals at the propagation layer. See STEM-GNN/results/encoder_lipschitz_reg.tex for that memo; its results are not merged into this paper’s tables since the two use different backbones, tasks, and baselines.

9 Adaptive Robustness Experiments

All values are mean $\pm$ standard deviation of node-classification accuracy (%) over 12 runs. Bold denotes the highest mean in each attacked column for each dataset.

9.1 Experiment 1: Semantic-Aware Adaptive Attack

We restrict the PGD attacker to the top- $p\%$ of the BM25 candidate pool, ranked by relevance score q_{ij} , with $p \in \{100, 75, 50, 25\}\%$. At $p=100\%$ the attacker has full access to all semantic candidates.

All methods at $p=100\%$. On Cora and WikiCS, v3 (Trust Budget) is the most robust method at every budget. On CiteSeer, H2GCN unexpectedly outperforms v3 across all attack budgets — a method with ego-network separation that appears to be harder to attack via semantic edge injection on this graph.

Method	Clean	5%	10%	20%	30%	40%
Cora						
GCN	83.0 $\pm$ 0.9	75.0 $\pm$ 0.9	70.8 $\pm$ 1.2	65.2 $\pm$ 2.0	61.5 $\pm$ 2.8	58.8 $\pm$ 3.6
GNNGuard	83.5 $\pm$ 0.9	75.5 $\pm$ 0.8	71.8 $\pm$ 1.0	66.8 $\pm$ 1.8	63.7 $\pm$ 2.8	61.8 $\pm$ 3.4
GCN-Jaccard	83.0 $\pm$ 0.9	75.0 $\pm$ 0.9	70.8 $\pm$ 1.2	65.2 $\pm$ 2.0	61.5 $\pm$ 2.8	58.8 $\pm$ 3.6
SCAD	83.7 $\pm$ 0.6	72.5 $\pm$ 2.1	65.1 $\pm$ 3.0	54.4 $\pm$ 3.9	46.8 $\pm$ 5.0	42.1 $\pm$ 5.0
APPNP	85.0 $\pm$ 0.5	74.9 $\pm$ 0.6	68.4 $\pm$ 0.9	58.4 $\pm$ 1.4	51.8 $\pm$ 2.2	47.1 $\pm$ 2.0
TWIRLS	83.8 $\pm$ 1.2	76.9 $\pm$ 1.3	73.4 $\pm$ 1.4	68.5 $\pm$ 1.7	65.7 $\pm$ 2.3	63.9 $\pm$ 2.9
H2GCN	81.4 $\pm$ 0.7	74.7 $\pm$ 0.8	71.7 $\pm$ 1.1	68.1 $\pm$ 2.0	65.9 $\pm$ 2.4	64.6 $\pm$ 2.6
GraphSAGE	81.4 $\pm$ 1.0	74.4 $\pm$ 1.2	71.0 $\pm$ 1.5	67.0 $\pm$ 2.7	64.5 $\pm$ 3.6	62.8 $\pm$ 4.0
GRAND	82.5 $\pm$ 0.9	73.9 $\pm$ 0.9	68.9 $\pm$ 0.9	61.3 $\pm$ 1.4	55.5 $\pm$ 2.0	50.9 $\pm$ 2.3
RUNG	83.5 $\pm$ 1.6	76.4 $\pm$ 1.6	72.9 $\pm$ 1.8	67.9 $\pm$ 2.2	64.5 $\pm$ 2.8	62.5 $\pm$ 3.4
v3 (Trust Budget)	82.0 $\pm$ 1.4	79.0 $\pm$ 1.6	77.2 $\pm$ 1.9	74.7 $\pm$ 2.3	72.9 $\pm$ 2.8	71.6 $\pm$ 2.9
CiteSeer						
GCN	74.2 $\pm$ 0.9	67.7 $\pm$ 1.0	64.0 $\pm$ 1.1	59.1 $\pm$ 1.3	55.8 $\pm$ 1.6	53.4 $\pm$ 2.0
GNNGuard	73.8 $\pm$ 1.1	67.3 $\pm$ 1.2	63.5 $\pm$ 1.5	58.6 $\pm$ 1.7	55.3 $\pm$ 2.0	53.0 $\pm$ 2.5
GCN-Jaccard	74.2 $\pm$ 0.9	67.7 $\pm$ 1.0	64.0 $\pm$ 1.1	59.1 $\pm$ 1.3	55.8 $\pm$ 1.6	53.4 $\pm$ 2.0
SCAD	73.5 $\pm$ 1.2	64.8 $\pm$ 1.3	60.1 $\pm$ 2.0	54.7 $\pm$ 2.8	50.5 $\pm$ 3.8	46.5 $\pm$ 4.6
APPNP	73.8 $\pm$ 1.1	65.8 $\pm$ 1.6	61.6 $\pm$ 2.7	55.8 $\pm$ 3.7	51.5 $\pm$ 4.5	48.4 $\pm$ 5.1
TWIRLS	74.0 $\pm$ 1.3	68.0 $\pm$ 1.3	64.8 $\pm$ 1.6	60.8 $\pm$ 2.0	58.3 $\pm$ 2.7	56.8 $\pm$ 3.3
H2GCN	72.6 $\pm$ 1.2	71.7 $\pm$ 1.0	71.3 $\pm$ 1.1	70.6 $\pm$ 2.4	70.3 $\pm$ 3.1	70.1 $\pm$ 3.4
GraphSAGE	72.3 $\pm$ 1.3	70.8 $\pm$ 2.3	70.0 $\pm$ 3.1	68.7 $\pm$ 5.3	68.1 $\pm$ 6.4	67.6 $\pm$ 7.3
GRAND	71.2 $\pm$ 1.0	69.5 $\pm$ 2.3	69.0 $\pm$ 2.9	66.2 $\pm$ 6.5	64.7 $\pm$ 8.8	63.4 $\pm$ 10.7
RUNG	74.4 $\pm$ 1.0	68.2 $\pm$ 1.3	64.9 $\pm$ 1.7	60.6 $\pm$ 2.4	57.9 $\pm$ 3.2	56.2 $\pm$ 4.1
v3 (Trust Budget)	73.3 $\pm$ 1.0	71.3 $\pm$ 1.0	70.1 $\pm$ 1.1	68.4 $\pm$ 1.1	67.0 $\pm$ 1.3	66.0 $\pm$ 1.2
WikiCS						
GCN	79.3 $\pm$ 0.7	61.3 $\pm$ 1.2	58.7 $\pm$ 1.3	56.4 $\pm$ 1.5	55.2 $\pm$ 1.3	54.6 $\pm$ 1.4
SCAD	78.4 $\pm$ 0.8	62.7 $\pm$ 1.5	58.5 $\pm$ 1.6	55.2 $\pm$ 2.0	53.8 $\pm$ 2.3	53.0 $\pm$ 2.5
APPNP	79.3 $\pm$ 0.6	66.2 $\pm$ 0.8	64.3 $\pm$ 1.0	62.4 $\pm$ 1.1	61.8 $\pm$ 0.9	61.2 $\pm$ 1.2
TWIRLS	79.3 $\pm$ 0.8	70.2 $\pm$ 0.9	68.3 $\pm$ 1.0	66.5 $\pm$ 1.2	66.1 $\pm$ 1.0	65.5 $\pm$ 1.1
RUNG	79.2 $\pm$ 0.8	70.2 $\pm$ 1.1	68.5 $\pm$ 0.9	66.9 $\pm$ 0.9	66.2 $\pm$ 1.2	65.8 $\pm$ 1.0
v3 (Trust Budget)	79.3 $\pm$ 0.7	75.8 $\pm$ 0.8	75.1 $\pm$ 0.7	74.3 $\pm$ 0.7	73.9 $\pm$ 0.7	73.7 $\pm$ 0.8

Pool-fraction sensitivity (RUNG and v3). Reducing the attack pool forces the attacker to use lower-quality candidate edges. RUNG recovers substantially as p decreases (e.g. Cora 62.5%→75.7% at $\epsilon=0.40$), while v3’s gain is more modest (71.6%→77.4%), confirming that v3’s trust budget makes it less dependent on pool quality.

Method	Clean	5%	10%	20%	30%	40%
Cora						
RUNG, $p=100\%$	83.5±1.6	76.4±1.6	72.9±1.8	67.9±2.2	64.5±2.8	62.5±3.4
RUNG, $p=75\%$	83.5±1.6	77.1±1.6	73.7±1.7	69.3±1.9	66.8±2.4	65.1±3.0
RUNG, $p=50\%$	83.5±1.6	78.2±1.5	75.7±1.6	72.8±1.9	71.2±2.0	70.1±2.1
RUNG, $p=25\%$	83.5±1.6	80.1±1.3	78.9±1.3	77.3±1.5	76.3±1.5	75.7±1.6
v3, $p=100\%$	82.0±1.4	79.0±1.6	77.2±1.9	74.7±2.3	72.9±2.8	71.6±2.9
v3, $p=75\%$	82.0±1.4	79.2±1.6	77.4±1.9	75.0±2.4	73.2±2.7	72.2±2.9
v3, $p=50\%$	82.0±1.4	79.6±1.5	78.2±1.7	76.3±2.1	75.1±2.4	74.2±2.5
v3, $p=25\%$	82.0±1.4	80.5±1.5	79.7±1.6	78.7±1.6	78.0±1.8	77.4±1.7
CiteSeer						
RUNG, $p=100\%$	74.4±1.0	68.2±1.3	64.9±1.7	60.6±2.4	57.9±3.2	56.2±4.1
RUNG, $p=75\%$	74.4±1.0	68.8±1.2	66.0±1.4	62.3±2.3	60.3±3.1	58.9±3.7
RUNG, $p=50\%$	74.4±1.0	69.9±1.2	67.6±1.4	65.2±2.1	63.9±2.5	63.1±2.7
RUNG, $p=25\%$	74.4±1.0	69.9±1.2	67.6±1.4	65.2±2.1	63.9±2.5	63.1±2.7
v3, $p=100\%$	73.3±1.0	71.3±1.0	70.1±1.1	68.4±1.1	67.0±1.3	66.0±1.2
v3, $p=75\%$	73.3±1.0	71.4±1.1	70.2±1.0	68.7±1.1	67.6±1.1	66.7±1.2
v3, $p=50\%$	73.3±1.0	71.6±1.0	70.7±1.0	69.5±1.1	68.6±1.1	68.1±1.2
v3, $p=25\%$	73.5±0.8	71.8±0.8	70.9±0.7	69.5±1.1	68.7±1.4	68.1±1.2
WikiCS						
RUNG, $p=100\%$	79.2±0.8	70.2±1.1	68.5±0.9	66.9±0.9	66.2±1.2	65.8±1.0
RUNG, $p=75\%$	79.1±0.6	71.9±0.8	70.6±1.0	69.5±0.9	68.6±0.9	68.2±1.1
RUNG, $p=50\%$	79.2±0.7	71.9±0.9	70.5±0.8	69.2±1.1	68.7±0.8	68.3±1.0
RUNG, $p=25\%$	79.2±0.8	75.1±0.8	74.2±0.7	73.7±0.9	73.3±1.0	73.3±1.0
v3, $p=100\%$	79.3±0.7	75.8±0.8	75.1±0.7	74.3±0.7	73.9±0.7	73.7±0.8
v3, $p=75\%$	79.2±0.6	75.9±0.7	75.3±0.7	74.7±0.6	74.4±0.6	74.2±0.7
v3, $p=50\%$	79.3±0.7	75.9±0.7	75.1±0.8	74.5±0.7	74.2±0.7	74.1±0.6
v3, $p=25\%$	79.2±0.8	76.7±0.9	76.4±0.7	76.1±0.7	75.8±0.7	75.9±0.8

9.2 Experiment 2: Scorer Quality Ablation

We fix $p=100\%$ and replace the BM25-derived q_{ij} with degraded scorers: LLM oracle (original BM25); Random ($q_{ij} \sim \mathcal{U}[0, 1]$); Noisy $\sigma \in \{0.5, 1.0, 2.0\}$ (BM25 plus $\mathcal{N}(0, \sigma^2)$ noise); Uniform-high (constant, disabling the budget).

On Cora and CiteSeer, the LLM oracle is the strongest scorer and robustness degrades gracefully with noise. Notably on Cora, Uniform-high achieves the highest robustness at $\epsilon=0.05$ (79.3% vs. LLM 79.0%), suggesting the trust-budget normalisation mechanism itself provides most of the benefit on this graph, independent of scorer quality. On WikiCS the LLM oracle wins at every budget. On PubMed, Random scoring slightly outperforms the oracle, confirming the normalised competition mechanism is the primary driver there.

Scorer	Clean	5%	10%	20%	30%	40%
Cora (victim: v3)						
LLM oracle	82.0±1.4	79.0±1.6	77.2±1.9	74.7±2.3	72.9±2.8	71.6±2.9
Random	82.7±1.2	78.1±1.1	75.3±1.4	71.4±1.7	68.9±2.2	67.1±2.5
Noisy $\sigma=0.5$	82.3±1.7	78.1±1.6	75.5±1.9	71.8±2.3	69.2±2.7	67.3±3.0
Noisy $\sigma=1.0$	82.0±1.7	77.7±1.9	74.9±2.3	71.2±3.0	68.6±3.4	67.0±4.1
Noisy $\sigma=2.0$	81.3±1.9	76.4±1.8	73.4±1.9	69.3±2.5	66.7±3.2	65.1±3.5
Uniform-high	83.2±1.2	79.3±1.4	77.0±1.7	73.7±2.2	71.6±2.4	70.2±2.5
CiteSeer (victim: v3)						
LLM oracle	73.3±1.0	71.3±1.0	70.1±1.1	68.4±1.1	67.0±1.3	66.0±1.2
Random	73.8±1.0	70.3±0.9	68.2±0.9	65.6±1.0	63.9±1.0	62.6±1.1
Noisy $\sigma=0.5$	73.9±0.7	71.4±0.7	69.9±0.7	68.0±0.6	66.6±0.6	65.6±0.6
Noisy $\sigma=1.0$	73.6±1.2	70.7±1.4	69.1±1.4	66.8±1.7	65.0±1.9	64.0±1.9
Noisy $\sigma=2.0$	73.4±1.1	70.0±1.0	67.8±1.4	65.0±2.2	62.8±2.4	62.4±1.4
Uniform-high	73.9±0.9	70.8±1.1	69.0±1.3	66.4±1.6	64.8±2.1	63.4±2.3
WikiCS (victim: v3)						
LLM oracle	79.3±0.7	75.8±0.8	75.1±0.7	74.3±0.7	73.9±0.7	73.7±0.8
Random	79.4±0.6	73.3±0.7	72.2±0.8	71.0±0.9	70.6±0.7	70.4±0.7
Noisy $\sigma=0.5$	79.4±0.6	75.5±0.7	74.4±0.7	73.8±0.7	73.4±0.8	73.3±0.8
Noisy $\sigma=1.0$	79.2±0.6	74.8±0.7	74.0±0.7	73.1±0.8	72.8±0.8	72.6±0.8
Noisy $\sigma=2.0$	79.3±0.7	74.2±0.8	73.3±0.7	72.5±0.8	71.8±0.9	71.7±0.9
Uniform-high	79.3±0.7	70.3±1.0	68.8±0.9	67.9±0.7	67.3±0.8	66.9±0.8
PubMed (victim: v3)						
LLM oracle	87.1±0.3	79.1±0.3	73.5±0.3	63.2±1.0	55.1±1.5	49.7±1.6
Random	87.1±0.3	79.7±0.3	74.4±0.4	64.8±1.0	57.2±1.4	51.7±1.6
Noisy $\sigma=0.5$	87.2±0.4	79.1±0.4	73.2±0.6	62.6±1.0	54.3±1.1	48.7±1.0
Noisy $\sigma=1.0$	87.1±0.3	79.0±0.4	73.3±0.6	62.6±1.0	54.3±1.1	48.8±1.0
Noisy $\sigma=2.0$	87.2±0.4	79.2±0.4	73.3±0.6	62.7±1.0	54.7±1.1	49.3±1.0
Uniform-high	87.1±0.3	78.6±0.3	72.4±0.4	60.8±1.0	52.6±1.1	47.4±1.1

9.3 Experiment 3: Transfer Attack

The attacker switches to a random- $k=20$ pool (20 uniformly sampled non-edges per node) while models retain their BM25-calibrated defences.

On Cora, v3 (76.2% at $\epsilon=0.05$) remains competitive with the best baseline (TWIRLS 76.3%) and outperforms RUNG (75.9%), unlike the WikiCS result where v3 falls below RUNG under the random pool. On CiteSeer, v3 (69.3%) is the most robust method at every budget. On WikiCS, RUNG maintains its advantage over v3 under the out-of-distribution pool, and GCN, SCAD, and APPNP all drop dramatically (e.g. GCN: 49.5% at $\epsilon=0.05$), far below their BM25-pool performance.

Method	Clean	5%	10%	20%	30%	40%
Cora						
GCN	83.0±0.9	75.0±0.9	70.8±1.1	65.2±1.9	61.5±2.7	58.8±3.5
GNNGuard	83.5±0.9	75.5±0.7	71.8±1.0	66.8±1.8	63.7±2.7	61.8±3.2
GCN-Jaccard	83.0±0.9	75.0±0.9	70.8±1.1	65.2±1.9	61.5±2.7	58.8±3.5
SCAD	83.7±0.6	72.5±2.0	65.1±2.9	54.4±3.8	46.8±4.8	42.1±4.8
APPNP	85.0±0.5	74.9±0.6	68.4±0.9	58.7±1.3	51.7±1.9	47.0±1.8
TWIRLS	83.8±1.2	76.9±1.2	73.4±1.4	68.5±1.6	65.7±2.2	63.9±2.8
H2GCN	81.4±0.7	74.7±0.8	71.7±1.1	68.1±1.9	65.9±2.3	64.6±2.5
GraphSAGE	81.4±0.9	74.4±1.2	71.0±1.5	67.0±2.6	64.5±3.4	62.8±3.8
GRAND	82.5±0.8	73.9±0.9	68.9±0.9	61.3±1.4	55.5±1.9	50.9±2.2
RUNG	83.5±1.5	76.4±1.5	72.9±1.7	67.9±2.1	64.5±2.6	62.5±3.3
v3 (Trust Budget)	82.0±1.3	79.0±1.5	77.2±1.8	74.7±2.2	72.9±2.7	71.6±2.7
CiteSeer						
GCN	74.2±0.8	67.7±0.9	64.0±1.0	59.1±1.3	55.8±1.5	53.4±1.9
GNNGuard	73.8±1.1	67.3±1.1	63.5±1.4	58.6±1.6	55.3±2.0	53.0±2.4
GCN-Jaccard	74.2±0.8	67.7±0.9	64.0±1.0	59.1±1.3	55.8±1.5	53.4±1.9
SCAD	73.4±1.1	64.8±1.3	60.1±1.9	54.7±2.7	50.5±3.6	46.5±4.4
APPNP	73.8±1.0	65.8±1.6	61.6±2.6	55.8±3.5	51.5±4.3	48.4±4.9
TWIRLS	74.0±1.2	68.0±1.3	64.8±1.6	60.8±2.0	58.3±2.6	56.8±3.2
H2GCN	72.6±1.2	71.7±0.9	71.3±1.1	70.6±2.3	70.3±3.0	70.1±3.3
GraphSAGE	72.3±1.3	70.8±2.2	70.0±3.0	68.7±5.1	68.1±6.2	67.6±7.0
GRAND	71.2±1.0	69.5±2.2	69.0±2.8	66.2±6.2	64.7±8.5	63.4±10.3
RUNG	74.4±1.0	68.2±1.2	64.9±1.6	60.6±2.3	57.9±3.1	56.2±3.9
v3 (Trust Budget)	73.3±0.9	71.3±1.0	70.1±1.0	68.4±1.1	67.0±1.2	66.0±1.1
WikiCS						
GCN	79.3±0.7	61.3±1.1	58.7±1.2	56.4±1.4	55.2±1.2	54.6±1.4
SCAD	78.5±0.7	62.7±1.4	58.5±1.5	55.2±1.9	53.8±2.2	53.0±2.3
APPNP	79.4±0.7	66.2±0.8	64.3±0.9	62.4±1.0	61.8±0.9	61.2±1.1
TWIRLS	79.2±0.8	70.2±0.8	68.3±1.0	66.5±1.1	66.1±1.0	65.5±1.0
RUNG	79.2±0.7	70.2±1.0	68.5±0.9	66.9±0.9	66.2±1.2	65.8±1.0
v3 (Trust Budget)	79.3±0.6	75.8±0.7	75.1±0.7	74.3±0.7	73.9±0.7	73.7±0.7